

SEGURIDAD EN APLICACIONES WEB

Clase #18 - 31 de agosto de 2026

Matías Corti - Martina García Améndola



DÓNDE QUEDAMOS

La clase pasada cerramos con esta deuda.
Qué pasa con ella hoy:

Pendiente de C17	Lo resuelve
Cualquiera borra cualquier libro. No hay usuarios	← HOY · modelo Usuario + JWT
No hay forma de guardar una contraseña	← HOY · bcrypt
No hay forma de saber quién hace el request	← HOY · authenticate
No hay diferencia entre un admin y un cliente	← HOY · authorize
Front sigue leyendo libros.json	C19 · front ↔ back
No hay pantalla de login	C19 y C21 · frontend

TEMAS DE HOY

- Seguridad: los tres principios, y qué vulnerabilidades ya no tienen
 - Autenticación vs autorización: 401 vs 403
 - Contraseñas: hash, salt, bcrypt - por qué un hash no se descripta
 - JWT: qué es, por qué no está encriptado, el flujo en 6 pasos
 - `authenticate` y `authorize`: dos middlewares, y por qué el orden importa
 - Práctica: `registro` → `login` → token → ruta protegida
-



PRINCIPIOS DE SEGURIDAD

*La seguridad no es una característica,
es un requisito fundamental*

Confidencialidad

→ proteger la información, sólo opera quién
corresponde

Integridad

→ asegurar que no se altere la información

Disponibilidad

→ que el sistema esté accesible
cuando se necesita

Los errores de seguridad no se ven hasta que es tarde. Lo
que se pierde: **datos, plata, reputación.**



CINCO

VULNERABILIDADES,
LAS TRES QUE YA
NO TENEMOS

Vulnerabilidad	Cómo se ve	Su estado
SQL Injection	' ; DROP TABLE "Libro"; --	✅ Cubierta. Prisma parametriza: manda el SQL y los datos por separado. Ese texto se guarda como título, no se ejecuta
XSS	<script>alert('hola')</script>	✅ Casi cubierta. React escapa todo lo que pasa por {}. Se rompe solo si usás dangerouslySetInnerHTML (el nombre te avisa)
Entradas sin validar	precio: -500 guardado	✅ Cubierta en C17. Zod en el middleware

El único caso en que volvéis a tener SQL injection:
`prisma.$queryRawUnsafe`. El nombre también te avisa.



LAS DOS QUE SÍ FALTAN

Vulnerabilidad	Cómo se ve hoy en su código	Se arregla
Exposición de datos sensibles	No hay tabla de usuarios. Cuando la creen, la contraseña se guarda como la escribió el usuario	HOY • hash + omit
Autenticación rota	No hay autenticación. DELETE /api/libros/1 lo puede hacer cualquiera desde cualquier terminal	HOY • JWT + middlewares

CSRF existe y hoy no aplica: no usamos cookies de sesión. Con un token en el header `Authorization`, el navegador no lo manda solo. Cuando en C21 discutamos dónde guardar el token, vuelve el tema.



MINI-CIERRE: SEGURIDAD

Concepto	En una línea
CIA	Confidencialidad, Integridad, Disponibilidad. Requisito, no feature
SQL injection	Cubierta por Prisma, porque parametriza. Salvo \$queryRawUnsafe
XSS	Cubierta por React, porque escapa. Salvo dangerouslySetInnerHTML
Validación	Cubierta en C17 con Zod. El front no protege: se saltea con curl
Lo que falta	Contraseñas guardadas en texto plano y ausencia de autenticación



AUTENTICACIÓN VS AUTORIZACIÓN

Dos preguntas distintas, en ese orden. La segunda no tiene sentido sin la primera.

	 AUTENTICACIÓN	 AUTORIZACIÓN
Pregunta	¿Quién sos?	¿Qué podés hacer?
Cómo	Email + contraseña → token	Se lee el rol del token
Resultado	Una identidad	Un permiso
Middleware	authenticate	authorize
Si falla	401 Unauthorized	403 Forbidden



401 VS 403

Status	Nombre	Significa	Cuándo
401	Unauthorized	"No sé quién sos"	Falta el token, está vencido, está mal firmado
403	Forbidden	"Sé quién sos, y no podés"	Token válido, pero el rol no alcanza

El **401** tiene un nombre que puede confundir: dice *Unauthorized* pero significa **no autenticado**. Es un error del estándar, de 1997, y ya no se puede cambiar.

Reintentar con otras credenciales sirve para un **401**. Para un **403** no sirve de nada: la identidad ya es correcta, el problema es el permiso.



DIFERENTES FORMAS DE AUTENTICAR

SESIONES (*Tradicional*)

- Cookie con Session ID
- Estado en el servidor
- Perfecto para monolitos

JWT (*Moderno*)

- Token autocontenido
- Stateless
- Ideal para APIs y microservicios

OAuth 2.0 (*Delegado*)

- "Login con Google/GitHub"
- No manejas passwords
- Delegar es más seguro

API Keys (*Simple*)

- Para servicios, no usuarios
- Una llave por aplicación



MATRIZ DE PERMISOS DE LA LIBRERÍA

Antes de escribir código, se decide en una tabla. Esto es lo que vamos a implementar hoy:

Endpoint	Invitado	CLIENTE	ADMIN
GET /api/libros	✓	✓	✓
GET /api/libros/:id	✓	✓	✓
POST /api/libros	✗ 401	✗ 403	✓
PUT /api/libros/:id	✗ 401	✗ 403	✓
DELETE /api/libros/:id	✗ 401	✗ 403	✓
GET /api/auth/yo	✗ 401	✓	✓

El catálogo es público: una librería que te pide login para mirar la vidriera no vende nada. Escribir requiere ser admin.



MINI-CIERRE: AUTENTICACIÓN Y AUTORIZACIÓN

Concepto	En una línea
Autenticación	¿Quién sos? → identidad → authenticate → falla con 401
Autorización	¿Qué podés? → permiso → authorize → falla con 403
El orden	Primero quién, después qué. Nunca al revés
Matriz de permisos	La decisión se toma en una tabla, antes del código
401 vs 403	"No sé quién sos" vs "sé quién sos y no podés"



HASH ≠ CIFRADO
≠ ENCODING

Tres cosas distintas que se confunden todo el tiempo.
Un hash no se descripta.

	Qué hace	¿Se puede volver?	Para qué sirve
Encoding (base64)	Cambia la representación	Sí, sin ninguna clave	Transportar datos. No es seguridad
Cifrado (AES)	Oculto con una clave	Sí, con la clave	Guardar algo que después necesitás leer
Hash (bcrypt)	Resume en algo irreversible	No, nunca	Guardar algo que nunca necesitás leer

Una contraseña entra en la tercera fila: el servidor **no necesita saber tu contraseña**. Solo necesita poder decir "sí, la que acabás de mandar es la misma".

```
"Libreria2026" → $2b$10$RbYWZ5cXCx7NiuxKQlNVd.ZtXFJz.cbeKM/fSPu.h/Fsaf.RXTdDS
                  |_____|
                  algoritmo costo salt (22 chars) el hash propiamente
```



EL SALT, Y POR QUÉ DOS CONTRASEÑAS IGUALES DAN HASHES DISTINTOS

El **salt** es un valor aleatorio que bcrypt genera y mezcla con la contraseña antes de hashear. Va guardado **dentro** del hash: no es un secreto.

```
await bcrypt.hash("Libreria2026", 10) // → $2b$10$RbYWZ5cXCx7NiuxKQlNVd.ZtXFJz...
await bcrypt.hash("Libreria2026", 10) // → $2b$10$xEdnB4l/r9.OIPfwxDrdyuIW4LWI...
//                                     └ mismo password, hash distinto
```

Sin salt, dos usuarios con la misma contraseña tendrían el mismo hash. Eso permite dos ataques: ver quién comparte contraseña, y precalcular una tabla gigante de hashes conocidos (*rainbow table*) una sola vez y usarla contra todas las bases del mundo.

Contra qué protege el hash: si te roban la base, no se llevan las contraseñas.

Contra qué NO protege: si la contraseña es 123456, el hash no te salva. Probar 123456 contra tu hash es una operación de 60 ms.



BCRYPT: LA API, Y POR QUÉ CONVIENE QUE SEA LENTO

```
import bcrypt from "bcrypt";

const hash = await bcrypt.hash("Libreria2026", 10); // al registrarse

const ok = await bcrypt.compare(intento, hash); // al loguearse → true/false
```

Dos funciones, las dos `async`. `compare` vuelve a hashear el intento con el salt que ya está dentro del hash guardado, y compara. **No descripta nada.**

El `10` es el **costo**: cada +1 incrementa exponencialmente el tiempo.

Costo	Tiempo por hash	
10	~60 ms	✅ lo que usamos
12	~235 ms	Producción con tráfico bajo
14	~950 ms	Un segundo por login. En una laptop lenta, tres

Ser lento es la función, no un defecto: un algoritmo rápido permite probar millones de contraseñas por segundo. 60 ms por intento arruina el negocio del que quiere probar a fuerza bruta.



DEMO: EL MISMO PASSWORD, DOS HASHES

En la terminal del container (de C18):

```
docker compose exec api node -e "  
const bcrypt = require('bcrypt');  
(async () => {  
  const a = await bcrypt.hash('Libreria2026', 10);  
  console.log(a);  
  const b = await bcrypt.hash('Libreria2026', 10);  
  console.log(b);  
  console.log('¿iguales?', a === b);  
  console.log('compare contra a:', await bcrypt.compare('Libreria2026', a));  
  console.log('compare contra b:', await bcrypt.compare('Libreria2026', b));  
  console.log('compare con otra:', await bcrypt.compare('otraCosa', a));  
})();
```



MINI-CIERRE: CONTRASEÑAS

Concepto	En una línea
Hash / cifrado / encoding	Irreversible / reversible con clave / reversible sin clave
Un hash no se desencrypta	El servidor no necesita saber tu contraseña, solo verificarla
Salt	Valor aleatorio por hash. Va adentro del hash. No es secreto
Por qué el salt	Mismo password → hashes distintos → no sirven las rainbow tables
Costo	Cada +1 duplica el tiempo. Lento es la función, no el defecto
Contra qué no protege	Contra una contraseña mala. 123456 se rompe en 60 ms



JWT: QUÉ ES Y QUÉ TIENE ADENTRO

Un estándar abierto (RFC 7519) para transmitir información verificable entre dos partes.

ESTRUCTURA

HEADER.PAYLOAD.SIGNATURE

Ejemplo:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWYWRtaW4iOiOnRydWUsIm1hdCI6MTc1NzE4NjUyN30.JJ8b9al392VJkJS02PxigZyuNm9w6weefa8flJ9er90

Parte	Qué tiene	Ejemplo
Header	Cómo está firmado	<code>{"alg":"HS256","typ":"JWT"}</code>
Payload	Quién es el usuario (claims)	<code>{"id":3,"rol":"CLIENTE","iat":...,"exp":...}</code>
Signature	La prueba de que nadie lo tocó	HMACHA256(header + "." + payload, SECRET)

`iat` = issued at, cuándo se emitió. `exp` = cuándo vence. Los pone `jsonwebtoken` solo.



UN JWT
NO ESTÁ
ENCRIPTADO.
ESTÁ FIRMADO

Copió el payload del token de la slide anterior y pegalo en la consola del navegador (F12):

```
atob("eyJpZCI6Mywicm9sIjoiQ0xJRU5URSI6Im1hdCI6MTc4Nzg1MTA3MH0")  
// → '{"id":3,"rol":"CLIENTE","iat":1787851070}'
```

Eso es base64: **encoding, no cifrado**. Cualquiera que tenga el token lee el payload. Sin clave, sin permiso, sin dejar rastro.

Lo que el secret garantiza no es el secreto: es que **nadie lo pueda modificar**. Cambiá un caracter del payload y la firma deja de coincidir.

De ahí sale la regla:

En el payload de un JWT no va nada que no quieras que el usuario lea. Ni contraseñas, ni hashes, ni datos personales, ni el DNI del cliente. Un **id** y un **rol** alcanzan.



6 minutos



JWT: VENTAJAS Y DESVENTAJAS

✓ Ventajas	⚠ Desventajas
Stateless: el server no guarda sesiones	Revocación difícil: no hay "cerrar sesión" de verdad
Escalable: funciona con varios servidores	Más grande: viaja entero en cada request
Portable: sirve para web, mobile y APIs	El secret es un punto crítico: si se filtra, se firman tokens
Autocontenido: trae los datos del usuario	Queda en el cliente: cuidado con localStorage y XSS
Independiente de dominio: CORS y microservicios	

Stateless significa que el servidor no recuerda nada: cada request trae su propia prueba de identidad. El costo es la fila de arriba a la derecha: si no guardás nada, **no tenés dónde anotar que un token dejó de valer**. Un token robado sirve hasta que vence.



EL FLUJO DE AUTENTICACIÓN, EN 6 PASOS

1. El front manda `POST /api/auth/login` con email y contraseña
2. El backend valida las credenciales y **genera el token** (†)
3. El front lo recibe y lo guarda
4. En cada request siguiente lo manda en el header `Authorization: Bearer <token>`
5. El middleware del backend **verifica la firma** y saca el usuario del payload
6. El backend responde con el recurso protegido (†)

(†) Solo si el paso anterior pasó la validación. Si no: `401`.

Hoy hacemos los pasos **2, 4** (a mano, con el `.http`), **5 y 6**.
Los pasos 3 y 4 desde React son C19 y C21.



MINI-CIERRE: JWT

Concepto	En una línea
Estructura	header.payload.signature, separados por puntos
Firmado ≠ encriptado	Cualquiera lo lee; nadie lo modifica sin el secret
Payload	id y rol. Nunca contraseñas ni datos sensibles
Cómo viaja	Header Authorization: Bearer <token>
Stateless	El server no guarda sesión. Costo: no se puede revocar
iat / exp	Emisión y vencimiento. Los pone la librería



AUTHENTICATE: ¿QUIÉN SOS?

```
// src/middlewares/auth.middleware.ts
export function authenticate(req: Request, res: Response, next: NextFunction) {
  const header = req.headers.authorization;
  if (!header?.startsWith("Bearer ")) {
    return res.status(401).json({ error: "Falta el token" });
  }
  try {
    const payload = jwt.verify(header.slice(7), JWT_SECRET) as PayloadToken;
    // lo deja para el resto de la cadena
    req.usuario = { id: payload.id, rol: payload.rol };
    next();
  } catch (error) {
    if (error instanceof jwt.TokenExpiredError) {
      return res.status(401).json({ error: "Token expirado" });
    }
    return res.status(401).json({ error: "Token inválido" });
  }
}
```

Error	Cuándo	Mensaje
TokenExpiredError	Pasó el exp	jwt expired
JsonWebTokenError	Firma inválida, o no es un token	invalid signature / jwt malformed

⚠ **jwt.decode() no verifica nada.** Lee el payload de cualquier token, incluso uno adulterado. Sirve para debuggear, **nunca** para autenticar. La función es **verify**.



REQ.USUARIO: EXTENDER EL TIPO DE REQUEST

`Request` de Express no tiene una propiedad `usuario`. Se la agregamos nosotros, con un archivo de declaración:

```
// src/types/express.d.ts
export {};

declare global {
  namespace Express {
    interface Request {
      usuario?: { id: number; rol: "ADMIN" | "CLIENTE" };
    }
  }
}
```

Si el archivo no está, o está fuera de lo que el `tsconfig.json` incluye:

```
error TS2339: Property 'usuario' does not exist on type
'Request<ParamsDictionary, any, any, ParsedQs, Record<string, any>>'.
```

Va **opcional** (?) porque en una ruta pública nadie lo llenó. En un controller protegido, `authenticate` ya corrió y podés confiar.

Nuestro `tsconfig.json` incluye toda la carpeta `src/` (con subcarpetas), así que un `.d.ts` dentro de `src/` entra solo. Fuera de ahí, TypeScript lo ignora **en silencio** y solo ves el TS2339.



AUTHORIZE: ¿PODÉS?

```
// src/middlewares/auth.middleware.ts
export function authorize(...roles: Array<"ADMIN" | "CLIENTE">) {
  return (req: Request, res: Response, next: NextFunction) => {
    if (!req.usuario) return res.status(401).json({ error: "No autenticado" });
    if (!roles.includes(req.usuario.rol)) {
      return res.status(403).json({ error: "No tenés permiso para esta operación" });
    }
    next();
  };
}
```

Mismo patrón que `validate` de C17: **una función que devuelve un middleware**. `authorize("ADMIN")` no autoriza nada, fabrica el middleware que autoriza para ADMIN.

```
router.delete("/:id", authenticate, authorize("ADMIN"), validateParams(idParamSchema),
  libroController.remove);
```

Los dos status, otra vez:

- Sin `req.usuario` → nadie lo autenticó → **401**
- Con `req.usuario` pero el rol no está en la lista → **403**



EL ORDEN DE LOS MIDDLEWARES

```
router.post("/",
  authenticate,           // 1. ¿Quién sos?      → 401
  authorize("ADMIN"),     // 2. ¿Podés?        → 403
  validate(libroCreateSchema), // 3. ¿El dato sirve? → 400
  libroController.create // 4. Hacelo          → 201
);
```

El orden no es estético. Cada paso corta la cadena si falla, y **los más baratos y los más restrictivos van primero:**

- **Autenticar antes de autorizar:** `authorize` lee `req.usuario`, que lo llena `authenticate`. Al revés, `req.usuario` está vacío y todo da 401.
- **Autorizar antes de validar:** no tiene sentido gastar CPU validando el body de alguien que no tiene permiso. Y peor: los mensajes de error de validación cuentan cosas sobre tu API a alguien que no debería estar ahí.

Verificado: `POST /api/libros` con token de ADMIN y `{"precio":-5}` devuelve **400**, no 403. Con token de CLIENTE y el body perfecto devuelve **403**, no 400.



MINI-CIERRE: MIDDLEWARES Y RUTAS PROTEGIDAS

Concepto	En una línea
authenticate	Lee el header, verifica la firma, llena req.usuario. Falla con 401
authorize(...roles)	Función que devuelve middleware. Falla con 403
req.usuario	Se declara en src/types/express.d.ts. Opcional, porque no siempre está
Orden	authenticate → authorize → validate → controller
verify vs decode	verify chequea la firma. decode solo lee: no autentica
Los dos errores	TokenExpiredError vs JsonWebTokenError. Los dos son 401



PRÁCTICA



PASO 0: CHEQUEO DE ARRANQUE

Ejemplo Completo:
DS26-Libreria (C18)

```
# 1) Todo arriba
docker compose up -d
docker compose ps      # api y db arriba, db "healthy"

# 2) Las dependencias de HOY, adentro del container
docker compose exec api npm ls bcrypt jsonwebtoken # ← si dice "empty", ver abajo

# 3) El cliente de Prisma, al día
docker compose exec api npx prisma generate

# 4) Los datos, ahí
docker compose exec db psql -U postgres -d libreria_db -c 'SELECT count(*) FROM "Libro";'
```

Listo cuando: los dos containers arriba, `bcrypt` y `jsonwebtoken` responden con versión, y el `SELECT` dice 10.

Si el paso 2 dice `(empty)`: faltó `docker compose build api`.
Instalarlo en el host **no** lo mete en la imagen.

 **5 minutos**



PASO 1: EL MODELO USUARIO Y EL ENUM ROL

Ejemplo Completo:
DS26-Libreria (C18)

```
// prisma/schema.prisma
enum Rol {
  ADMIN
  CLIENTE
}

model Usuario {
  id          Int          @id @default(autoincrement())
  email       String       @unique // ← @unique: sin cuentas con el mismo mail
  passwordHash String      // ← el NOMBRE avisa qué se guarda acá
  nombre      String
  rol         Rol          @default(CLIENTE) // ← nadie se registra como admin
  creadoEn    DateTime @default(now())
}
```

Tres decisiones que valen más que el código:

- **passwordHash, no password.** El nombre de la columna documenta que ahí no hay una contraseña.
- **@default(CLIENTE).** El rol no lo elige quien se registra. Si estuviera en el body, cualquiera se hace admin.
- **enum en vez de String.** La base rechaza "admin" en minúscula o "ADMIM". Un typo deja de ser un agujero de seguridad.

En C17 usamos **Usuario** ↔ **Perfil** como ejemplo hipotético de 1:1. Hoy **Usuario** es real.

 **7 minutos**



PASO 2: LA MIGRACIÓN, Y EL COMANDO QUE NO SE DISPARA SOLO

[Ejemplo Completo:](#)
[DS26-Libreria \(C18\)](#)

```
# Renombramos Categoría → Categoria: es un rename de tabla. En desarrollo,
reset.
docker compose exec api npx prisma migrate reset --force

docker compose exec api npx prisma migrate dev --name usuarios_y_rols
docker compose exec api npx prisma generate # APARTE. migrate dev NO lo dispara
docker compose exec api npx prisma db seed # APARTE. reset NO corre el seed
```

Si te salteás `prisma generate`:

```
error TS2339: Property 'usuario' does not exist on type 'PrismaClient<...>'.
```

El modelo está en el schema y la tabla está en la base. Lo que falta es el cliente de TypeScript, que es un archivo generado y no está en Git (`src/generated/` está en `.gitignore`).

La regla, otra vez: `prisma generate` después de **todo** cambio de schema, y después de **todo** `git switch`.

 **5 minutos**



PASO 3: EL SEED CON CONTRASEÑAS HASHEADAS

[Ejemplo Completo:](#)
[DS26-Libreria \(C18\)](#)

En Producción **nunca** una contraseña en texto plano en la base.
Tampoco en el seed.

```
// prisma/seed.ts
const usuarios = [
  { email: "admin@libreria.test", nombre: "Admin", rol: "ADMIN" as
const, password: "Admin1234" },
  { email: "cliente@libreria.test", nombre: "Cliente", rol: "CLIENTE" as
const, password: "Cliente1234" },
];

for (const { password, ...datos } of usuarios) { // el password se saca del
spread
  await prisma.usuario.upsert({
    where: { email: datos.email }, // upsert = idempotente: corré dos veces
    update: {},
    create: { ...datos, passwordHash: await bcrypt.hash(password, 10) },
  });
}
```

⚠ **Y el bug que compiló igual:** la primera versión era `create: { ...u, passwordHash: ... }`. El spread arrastra `password`, `tsc` no dice nada, y Prisma revienta en runtime:

Unknown argument `password`. Available options are marked with ?.

TypeScript no chequea propiedades sobrantes cuando vienen de un spread. Por eso se desestructura.



5 minutos



OMIT:
QUE EL HASH
NO SE ESCAPE
POR ACCIDENTE

Un `findMany` devuelve **todas** las columnas. Incluida `passwordHash`.
Un solo endpoint distraído y el hash de todos los usuarios sale en un JSON.

```
// config/prisma.ts - una vez, para toda la app
export const prisma = new PrismaClient({
  adapter,
  omit: { usuario: { passwordHash: true } }, // ← seguro por defecto
});
```

Ahora `prisma.usuario.findUnique(...)` **nunca** trae el hash, y el tipo tampoco lo tiene: si escribís `usuario.passwordHash`, no compila.

La excepción: el login sí lo necesita, y hay que pedirlo explícito.

```
const usuario = await prisma.usuario.findUnique({
  where: { email: datos.email },
  omit: { passwordHash: false }, // ← acá SÍ lo usa. Sino: compare(pass, undefined)
});
```



PASO 4: LA CONFIG

[Ejemplo Completo:](#)
[DS26-Libreria \(C18\)](#)

```
# .env      y en .env.example con placeholder
JWT_SECRET=cambiame_por_una_cadena_larga_y_aleatoria
```

```
// src/config/env.ts
import type { SignOptions } from "jsonwebtoken";

function obligatorio(nombre: string): string {
  const valor = process.env[nombre];
  if (!valor) throw new Error(`Falta ${nombre} en el .env`); // falla al ARRANCAR
  return valor;
}

export const JWT_SECRET      = obligatorio("JWT_SECRET");
export const JWT_EXPIRES_IN: SignOptions["expiresIn"] = "2h"; // constante, NO va al
.env
export const SALT_ROUNDS     = 10;
```

 **6 minutos**



PASO 5: LOS SCHEMAS DE ZOD PARA AUTH

Ejemplo Completo:
DS26-Libreria (C18)

```
// src/validations/auth.validation.ts
import { z } from "zod";

const email = z.string().trim().toLowerCase().pipe(z.email("Email inválido"));

export const registroSchema = z.object({
  nombre: z.string().trim().min(1, "El nombre es obligatorio").max(100),
  email,
  password: z.string()
    .min(8, "La contraseña necesita al menos 8 caracteres")
    .regex(/[A-Z]/, "Necesita al menos una mayúscula")
    .regex(/[0-9]/, "Necesita al menos un número"),
});

export const loginSchema = z.object({
  email,
  password: z.string().min(1, "La contraseña es obligatoria"), // ← acá NO se valida fortaleza
});
```

Tres detalles que importan:

- **z.email()**, no **z.string().email()**. La segunda funciona pero está deprecada en Zod 4.
- **.trim().toLowerCase()** antes del email. **z.email()** no normaliza: "ANA@Libreria.TEST" fallaría, y Ana@x y ana@x serían dos cuentas distintas.
- **El login no valida fortaleza.** Si mañana subís el mínimo a 10 caracteres y el login lo exige, todos los usuarios viejos quedan afuera. El login solo pregunta si el campo está.
- **El rol no está en ningún schema.** Lo pone la base con `@default(CLIENTE)`.



6 minutos



PASO 6: REGISTRO HASHEAR Y GUARDAR

```
// src/services/auth.services.ts
export async function registrar(datos: Registro): Promise<UsuarioPublico> {
  const hash = await bcrypt.hash(datos.password, SALT_ROUNDS);
  return prisma.usuario.create({
    data: { nombre: datos.nombre, email: datos.email, passwordHash: hash },
    select: { id: true, email: true, nombre: true, rol: true }, // nunca el hash
  });
}
```

```
// src/controllers/auth.controller.ts
export async function registrar(req: Request, res: Response) {
  const usuario = await authService.registrar(req.body);
  return res.status(201).json(usuario);
}
```

Email repetido: no hace falta preguntar antes. La base tiene @unique, Prisma tira **P2002** y el **errorHandler** de C17 ya lo traduce a **409**. Cero líneas nuevas.

```
POST /api/auth/registro → 409 { "error": "Ya existe un registro con ese valor" }
```



PASO 7: LOGIN COMPARAR Y FIRMAR

Ejemplo Completo:
DS26-Libreria (C18)

```
// src/services/auth.services.ts
export async function login(datos: Login) {
  const usuario = await prisma.usuario.findUnique({
    where: { email: datos.email },
    omit: { passwordHash: false }, // el omit global lo esconde: acá lo necesito
  });
  if (!usuario) return null;

  const coincide = await bcrypt.compare(datos.password, usuario.passwordHash);
  if (!coincide) return null; // ← mismo return que arriba, a propósito

  const payload = { id: usuario.id, rol: usuario.rol };
  const token = jwt.sign(payload, JWT_SECRET, { expiresIn: JWT_EXPIRES_IN });
  return { token, usuario: { id: usuario.id, email: usuario.email, nombre:
    usuario.nombre, rol: usuario.rol } };
}
```

```
// controller
if (!resultado) return res.status(401).json({ error: "Credenciales inválidas" });
```

Un solo mensaje para los dos casos, y a propósito. Si el mail no existiera dijera "usuario no encontrado" y la contraseña mal dijera "contraseña incorrecta", cualquiera podría probar mails hasta descubrir **qué mails están registrados**. Se llama *user enumeration*. El mensaje genérico cuesta cero y lo cierra.

 8 minutos



PASO 8: LAS RUTAS DE AUTH

[Ejemplo Completo:](#)
[DS26-Libreria \(C18\)](#)

```
// src/routes/auth.routes.ts
router.post("/registro", validate(registroSchema), authController.registrar);
router.post("/login",    validate(loginSchema),    authController.login);
router.get( "/yo",        authenticate,            authController.yo);
```

```
// src/index.ts
app.use(express.json());

app.use("/api/auth",    authRoutes);           // ← NUEVO
app.use("/api/libros",  libroRoutes);
app.use("/api/autores", autorRoutes);

app.use(errorHandler);                         // ← siempre último
```

registro y **login** son **públicas**: si pidieran token, no habría forma de conseguir el primero.

 **4 minutos**



PASO 9: PROBARLO CON EL .HTTP, CAPTURANDO EL TOKEN

[Ejemplo Completo:](#)
[DS26-Libreria \(C18\)](#)

REST Client guarda la respuesta de un request con nombre y podés usarla en los siguientes:

```
@baseUrl = http://localhost:3000

### Registro
POST {{baseUrl}}/api/auth/registro
Content-Type: application/json

{ "nombre": "Ana", "email": "ana@libreria.test", "password": "Libreria2026" }

### Login ← el @name es lo que permite capturar la respuesta
# @name login
POST {{baseUrl}}/api/auth/login
Content-Type: application/json

{ "email": "ana@libreria.test", "password": "Libreria2026" }

### Guardamos el token de la respuesta anterior
@token = {{login.response.body.token}}

### Ahora cualquier request protegido lo usa
GET {{baseUrl}}/api/auth/yo
Authorization: Bearer {{token}}
```

Listo cuando: el registro devuelve 201 sin el hash en la respuesta, el login devuelve 200 con token, y el 409 sale al registrar el mismo mail dos veces.

 **5 minutos**



PASO 10: PROTEGER LOS ENDPOINTS

Ejemplo Completo:
DS26-Libreria (C18)

Implementen la matriz de la slide 10. Lectura pública, escritura de admin.

```
// src/routes/libro.routes.ts
router.get("/", libroController.getAll); // público

router.get("/:id", validateParams(idParamSchema), libroController.getById); // público

router.post("/", authenticate, authorize("ADMIN"), validate(libroCreateSchema), libroController.create);

router.put("/:id", authenticate, authorize("ADMIN"), validateParams(idParamSchema), validate(libroUpdateSchema), libroController.update);

router.delete("/:id", authenticate, authorize("ADMIN"), validateParams(idParamSchema), libroController.remove);
```

Y el endpoint que usa `req.usuario`:

```
// GET /api/auth/yo
export async function yo(req: Request, res: Response) {
  const usuario = await authService.findById(req.usuario!.id); // authenticate ya corrió
  if (!usuario) return res.status(404).json({ error: "Usuario no encontrado" });
  return res.json(usuario);
}
```

Lo mismo en `autor.routes.ts`.



8 minutos



PASO 11: LA TABLA DE PRUEBAS

[Ejemplo Completo:](#)
[DS26-Libreria \(C18\)](#)

Corran el `.http` y verifiquen los doce casos. Esto es la matriz de la slide 10, ejecutada.

#	Request	Token	Esperado
1	POST /auth/registro	-	201 sin passwordHash en la respuesta
2	POST /auth/registro mismo email	-	409
3	POST /auth/registro password "corta"	-	400 con los 3 errores del password
4	POST /auth/login mail que no existe	-	401 "Credenciales inválidas"
5	POST /auth/login password mal	-	401 el mismo mensaje
6	POST /auth/login correcto	-	200 con token
7	GET /auth/yo	sin header	401
8	GET /auth/yo	CLIENTE	200
9	POST /api/libros	sin header	401
10	POST /api/libros	CLIENTE	403
11	POST /api/libros	ADMIN	201
12	GET /api/libros	sin header	200 - es público

Listo cuando: los doce dan lo de la última columna.



7 minutos



ROMPÉ EL TOKEN

Tomá tu token del login y cambiale **un solo caracter del payload** (la parte del medio). Mandalo:

```
GET {{baseUrl}}/api/auth/yo  
Authorization: Bearer eyJhbGci....eyJpZCI6OTk5...MODIFICADO...NhRx35Lo...
```

```
401 { "error": "Token inválido" }
```

Ahora probá lo mismo pero editando el payload para poner **"rol": "ADMIN"** y pegale a **POST /api/libros**. También **401**: no llega ni a la parte del rol.

Eso es la firma trabajando. Podés **leer** el payload pero no podés **cambiarlo**: cambiar un byte invalida la firma, y recalcularla necesita el **JWT_SECRET**, que solo tiene el servidor.



5 minutos



¿TERMINASTE? EXTENSIONES

Opcional. El piso son los doce casos del Paso 12.

1. **PUT /api/auth/yo** para que cada usuario edite su propio nombre. Si manda `rol`, se ignora.
2. **Que un CLIENTE pueda ver solo lo suyo.** `GET /api/auth/yo` ya lo hace; pensá qué otro endpoint del TP necesitaría la misma lógica.
3. **Expiración corta:** poné `JWT_EXPIRES_IN = "10s"`, logueate, esperá y volvé a pegarle. Verificá que el mensaje sea "Token expirado" y no "Token inválido".
4. **Difícil, y es conceptual:** hoy `authorize("ADMIN")` decide por rol. ¿Cómo harías para que un CLIENTE pueda editar **su propio** pedido pero no el de otro? El rol no alcanza: hay que comparar `req.usuario.id` con el dueño del recurso. ¿En qué capa va esa pregunta, y por qué no puede ir en el middleware genérico?



MINI-CIERRE: LA PRÁCTICA

Lo que hicimos	En una línea
Bugs de C17	El errorHandler montado, el if con llaves, Categoría sin tilde
Usuario + Rol	email @unique, passwordHash, rol con default. El rol no viene del cliente
Seed	Contraseñas hasheadas, upsert para que sea idempotente
omit global	El hash no sale por accidente, y el tipo tampoco lo tiene
registro / login	hash al crear, compare al entrar, sign para el token
authenticate / authorize	401 y 403, en ese orden, antes de validar
api.http	Doce casos, y el token capturado con @name



LO QUE FALTA, FALTA A PROPÓSITO

Hoy nos llevamos usuarios de verdad, contraseñas que no se pueden leer, y endpoints que saben quién les está hablando.
Lo que falta, y por qué:

Pendiente	Qué es	Por qué hoy no
Refresh tokens	Un token corto + otro largo para renovarlo	Duplica el flujo. Con 2h alcanza para la cursada
Rate limiting en login	Frenar al que prueba 10.000 contraseñas	Una librería más y un middleware. No cambia lo conceptual
HTTPS	Sin él, el token viaja en texto plano	Es del deploy, no del código. C22
Cookies httpOnly vs localStorage	Dónde guardar el token en el navegador	Es una decisión del front. C21
Recuperar contraseña	El mail con el link	Necesita mandar mails
2FA	Segundo factor	Existe, aunque excede el alcance de la materia



¿Y EL FRONT? EL CONTRATO QUEDA DEFINIDO HOY

Todo lo de hoy es backend. El front lo tocamos en las dos clases siguientes, y el contrato es este:

```
POST /api/auth/login
  → 200 { "token": "eyJ...", "usuario": { "id": 3, "rol": "CLIENTE" } }

GET /api/libros
  → 200 (público, sin header)

POST /api/libros
Authorization: Bearer eyJ...
  → 201 ( sin header: 401 - rol CLIENTE: 403 )
```

Qué falta	Dónde
Pantalla de login, guardar el token, mandarlo en cada fetch	C19 · Integración front ↔ back
AuthContext, PrivateRoute, esconder botones según el rol	C21 · Autorización en el frontend

⚠ **Entrega 3 es el 13/09** y pide backend + DB + **auth integrado con el frontend**. C19 es la clase de integración: si el auth del backend no está andando, no es posible integrar.

ACTIVIDADES

ACTIVIDAD INDIVIDUAL

1° instancia hasta el
06/09/2026

2° instancia: **13/09/2026**
(Cierre Bloque 3)

[AI-C18] Seguridad en la Librería

Referencia en el repo de ejemplo [branch C18](#)

En `ds2026-actividades`: copiar el sitio de la librería de la Clase 17 en `ejercicios/c18-autenticacion/`

1. `schema.prisma` con el modelo `Usuario` (`email @unique`, `passwordHash`, `nombre`) y el enum `Rol` con default
2. Migración commiteada, con nombre descriptivo
3. `seed.ts` con al menos un ADMIN y un CLIENTE, **contraseñas hasheadas**, y que corra dos veces sin explotar
4. `validations/auth.validation.ts` con `registroSchema` y `loginSchema`. El login **no** valida fortaleza
5. `POST /api/auth/registro` → 201 sin el hash en la respuesta, 409 si el mail se repite
6. `POST /api/auth/login` → 200 con token, 401 con **mensaje genérico** en los dos casos de falla
7. `middlewares/auth.middleware.ts` con `authenticate` y `authorize`, y `src/types/express.d.ts`
8. Escritura de libros y autores solo para ADMIN. Lectura pública
9. `GET /api/auth/yo` usando `req.usuario`
10. `api.http` con los doce casos del Paso 12, incluyendo el token capturado con `@name`
11. `.env.example` actualizado con `JWT_SECRET` **con placeholder**. El `.env` no va a Git
12. 10. Commiteá y avisá en `#actividades`
→ hilo **[AI-C18] Seguridad en la Librería**

ACTIVIDAD GRUPAL

Hasta el **06/09/2026**
(inclusive)



RECORDATORIO

El **13/09/2026** es la
ENTREGA 3 (Backend) del
Trabajo Práctico Grupal

[AG-C18] Seguridad en el TP

1. **Matriz de permisos** del TP, por endpoint, en la carpeta de Documentación. Los roles que su dominio necesite, no copiar ADMIN/CLIENTE si no aplica
2. Modelo de usuario en `schema.prisma` con los roles de su dominio, migración commiteada
3. `seed.ts` con un usuario por rol, contraseñas hasheadas
4. `registro` y `login` andando, con los status de la matriz
5. `authenticate` y `authorize` aplicados a **todos** los endpoints según la matriz
6. `api.http` con los casos de error de sus entidades: `401`, `403`, `409`
7. **Repartir el trabajo:** al menos un PR con review cruzado por integrante. Mover las tarjetas de Trello a "Done"
8. Retrospectiva del sprint y planning del siguiente, con el tablero actualizado
9. Aviso en `#actividades`
→ hilo **[AG-C18] Seguridad en el TP**



ANTES DE LA PRÓXIMA CLASE

⚠ INSTALAR ANTES DE C19

El aula no tiene internet confiable.

Sin instalación = sin práctica.

```
bash

# 1) Traer el ejemplo de hoy y regenerar el cliente
git pull
cd backend
docker compose exec api npx prisma generate

# 2) La dependencia de C19: CORS (el navegador va a entrar en juego)
npm install cors
npm install -D @types/cors

# 3) Reconstruir la imagen ← este paso NO es opcional
cd ..
docker compose build api

# 4) Verificar que quedó adentro del contenedor
docker compose up -d
docker compose exec api npm ls cors
```

✓ Dudas → canal #consultas de Discord

July 17

C19: Integración Front ↔ Back (07/09/2026)